

Rule-Based Campus Help Chatbot

GenAI Course – Practical Assessment

Student Name	Shaik Latheef
Register Number	99230040764
Department	CSE
Programming Language	Python

1. Project Objective

The project develops a small Python-based campus help chatbot that responds to common student requests through predefined intent rules. Instead of using an online AI service, the program identifies useful keywords in the user's message and selects a suitable response. The objective is to demonstrate a simple conversational workflow using core Python features.

2. Proposed Features

Feature	Description
User greeting	Starts the session with a personalized welcome.
Campus information	Answers basic questions about classes, library and labs.
Study support	Suggests simple study practices and revision steps.
Command display	Shows the types of questions the chatbot can handle.
Message matching	Uses keyword groups to classify common requests.
Exit control	Ends the conversation when the user enters an exit command.
Fallback handling	Responds safely when the message is outside the supported topics.

3. Python Implementation

```
def campus_helper():
    name = input("Student name: ").strip()
    print(f"Bot: Welcome, {name}. How can I help today?")

    while True:
        query = input("You: ").strip().lower()

        if query in {"exit", "end", "goodbye"}:
            print("Bot: Thanks for chatting. Have a productive day!")
            break

        if any(k in query for k in ("hello", "hi", "good morning")):
            print("Bot: Hello! You can ask about campus services or study support.")

        elif any(k in query for k in ("library", "books", "reading")):
            print("Bot: The library is useful for textbooks, reference material and quiet study.")

        elif any(k in query for k in ("lab", "laboratory", "practical")):
            print("Bot: Practical sessions are supported through the department computer labs.")

        elif any(k in query for k in ("class", "timetable", "lecture")):
```

```

        print("Bot: Please check your current class timetable for the exact room and schedule.")

    elif any(k in query for k in ("study", "exam", "revision")):
        print("Bot: Break the syllabus into topics, revise regularly, and practise questions.")

    elif any(k in query for k in ("help", "options", "services")):
        print("Bot: I can discuss library, labs, classes, study tips, or exit commands.")

    else:
        print("Bot: I don't have an answer for that yet. Try asking about library, labs, classes or study.")

if __name__ == "__main__":
    campus_helper()

```

4. Working Principle

The chatbot first obtains the student's name and then repeatedly accepts messages. Each message is trimmed and converted to lowercase so that differences in capitalization do not affect matching. Exit commands are handled before other categories. For normal messages, the program checks several keyword groups representing campus and study-related intents. The first matching group determines the response. If no group matches, a fallback response directs the student toward supported topics.

5. Sample Test Cases

User input	Observed result
hello	Displays a greeting and available topic areas.
Where can I find books?	Provides a library-related response.
Tell me about the lab	Returns information about practical lab support.
How should I revise for exams?	Gives a short study suggestion.
What services do you support?	Lists the chatbot's supported categories.
something unrelated	Shows the fallback response.
goodbye	Closes the chatbot session.

6. Learning Outcomes

This project provides practical experience with Python input/output, string processing, sets, loops, conditional statements, functions and formatted strings. It also demonstrates a basic approach to intent classification without depending on an external AI model. The exercise shows how a structured rule system can turn ordinary text input into predictable conversational responses.

7. Conclusion

The completed chatbot is a compact example of a conversational interface built with standard Python. Although its knowledge is intentionally limited, its modular keyword rules make the behaviour easy to test and extend. Additional intents could be added later for other student-support questions.